



浙江大学
ZHEJIANG UNIVERSITY



浙江大学 区块链与数据安全
全国重点实验室
STATE KEY LABORATORY OF BLOCKCHAIN AND DATA SECURITY
ZHEJIANG UNIVERSITY

FICTI  N

Inria

Formalizing the Linux eBPF Core ISA

A Mechanized Operational Semantics and Its Real-World Applications

Shenghao Yuan^{1,*}, **Yazhou Tang**^{1,*}, Tianci Cao¹,
Frédéric Besson², Jean-Pierre Talpin², Mingshuai Chen^{1,†}

¹Zhejiang University, China ²INRIA, France

Presenter: Zhongyi Wang on behalf of **Yazhou Tang**

OOPSLA 2026 · Oakland, CA, USA · October 6, 2026

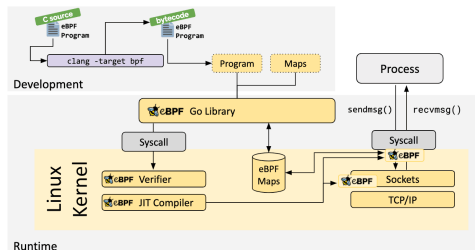


What is eBPF?



extended **B**erkeley **P**acket **F**ilter

- Merged into Linux 3.18 (2014).
- Runs sandboxed programs at kernel hooks.
- **Safely** and **efficiently** extends the kernel—without kernel patches or modules.

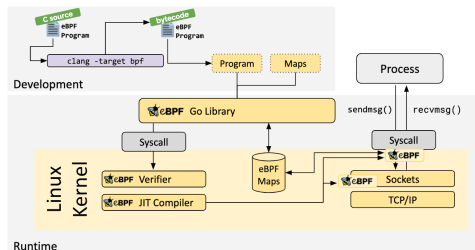




What is eBPF?

extended **B**erkeley **P**acket **F**ilter

- Merged into Linux 3.18 (2014).
- Runs sandboxed programs at kernel hooks.
- **Safely** and **efficiently** extends the kernel—without kernel patches or modules.



Representative applications



Cilium
K8s networking
& load balancing



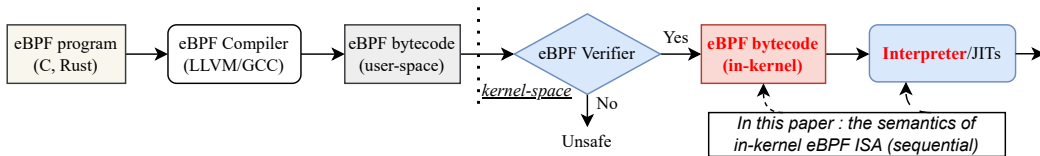
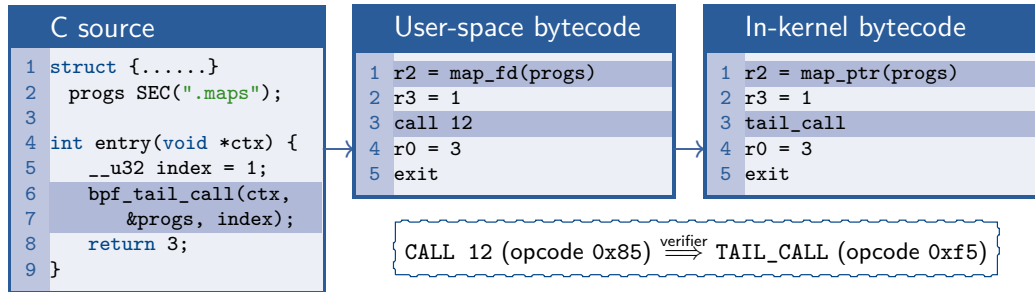
bpftool
Dynamic tracing
& performance analysis



Falco
Runtime security
& anomaly detection



eBPF: Example and Pipeline



Motivation



Verification needs semantics

① Verifier analysis

Prevail (PLDI '19), TNum
(CGO '22), Agni (CAV '23)

② Proof-carrying code

Exoverifier (LPC '21), VEP
(NSDI '25), BCF (SOSP '25)

③ JITs & Compilers

Jitterbug (OSDI '20), JitSynth
(CAV '20), BeePL (PriSC '25)

Motivation



Verification needs semantics

① Verifier analysis

Prevail (PLDI'19), TNum (CGO'22), Agni (CAV'23)

② Proof-carrying code

Exoverifier (LPC'21), VEP (NSDI'25), BCF (SOSP'25)

③ JITs & Compilers

Jitterbug (OSDI'20), JitSynth (CAV'20), BeePL (PriSC'25)

Our semantic reference

• ISA documentation

describes the expected instruction behavior in natural language.

• Linux eBPF interpreter

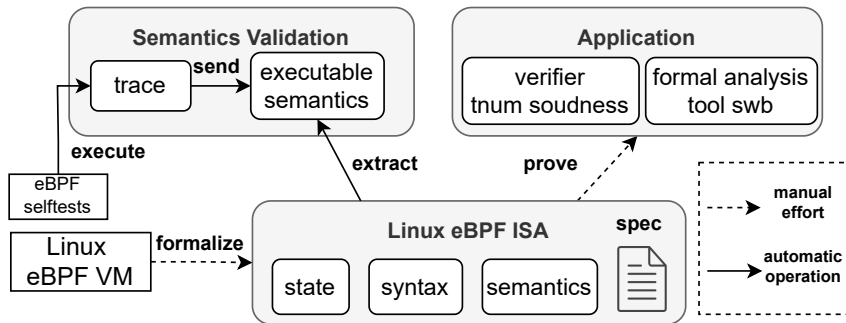
realizes that behavior and serves as our reference implementation.

Goal: a **shared, complete, and validated** formal semantics of eBPF.

Overview of our work



- 1 **ISA mechanization:** An executable Rocq semantics covering all **153 sequential instructions**.
- 2 **Linux interpreter validation:** An extracted interpreter checked **step by step** against kernel traces.
- 3 **Applications:** `tnum` soundness proofs, bug findings, and **upstreamed improvements**.





Mechanizing the Linux eBPF ISA

Small-Step Operational Semantics

$$\mathcal{S} \xrightarrow{ins} \mathcal{S}'$$

Program state

$$\mathcal{S} = \langle \mathcal{R}, \mathcal{M}, STK, cnt \rangle$$

$\mathcal{R}$	Registers r_0 – r_{11} and pc (CompCert values)
$\mathcal{M}$	CompCert memory
STK	Internal-call stack frames
cnt	Tail-call counter

Additional static inputs

- Main program
- Tail-call target programs
- Accessible regions

Initial and final states

Initial	Final
$pc \mapsto$ first instruction	$pc = \text{nullptr}$
$r_1 \mapsto$ program context	$r_0 = \text{result}$
$r_{10} \mapsto$ stack top	$r_{10} \mapsto$ stack top
$STK = [], cnt = 0$	$STK = []$

Outputs

- Executable Rocq semantics for **all 153 sequential instructions**¹
- A **manually curated specification** mapping instructions to their semantics

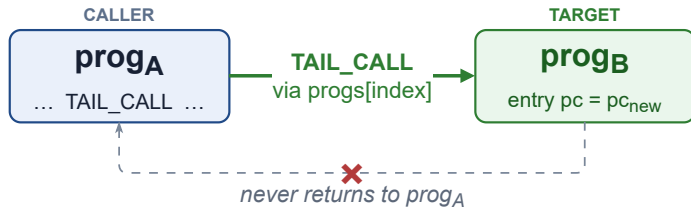
¹Based on Linux v7.0-rc3.



Example: eBPF Tail Call

Concrete setup

r_2	$\&\text{progs}$
r_3	index
$\text{progs}[\text{index}]$	pc_{new}
cnt	0



Success (Jump to $Prog_B$)

$$\mathcal{S} \xrightarrow{\text{TailCall}} \mathcal{S}[pc \leftarrow pc_{\text{new}}, cnt \leftarrow cnt + 1]$$

Fallback (Remain in $Prog_A$)

$$pc_{\text{new}} = \text{nullptr} \vee \text{max_call} \leq \text{index} \vee 33 \leq cnt \implies \mathcal{S} \xrightarrow{\text{TailCall}} \mathcal{S}[pc^{+1}]$$

²See JMP_TAIL_CALL in Linux eBPF interpreter and exec_tail_call in Rocq model.

Modeled Linux eBPF instructions



Type	Instructions	Count
ALU32/ALU64	K/X: ADD, SUB, MUL, DIV, SDIV, OR, AND, LSH, RSH, MOD, SMOD, XOR, MOV, ARSH; NEG, MOVSX, LE/BE, BSWAP	72
Jumps/calls	JMP/JMP32 K/X: JEQ, JNE, JGT, JLT, JGE, JLE, JSET, JSGT, JSLT, JSGE, JSLE; JA, helper/kfunc/local calls, tail call, EXIT	51
Memory	LDX_MEM_B/H/W/DW, ST_MEM_B/H/W/DW, STX_MEM_B/H/W/DW, LDX_MEMSX_B/H/W	15
Probe loads	LDX_PROBE_MEM_B/H/W/DW, LDX_PROBE_MEMSX_B/H/W	7
Wide immediates	LD_IMM64, LD_MAP_FD, LD_MAP_VALUE_FD, LD_BTFFID, LD_FUNC, LD_MAP_IDX, LD_MAP_VALUE_IDX	7
Barrier	ST_NOSPEC	1
Total	Modeled sequential instructions	153

Notes:

- 1 Cover **all sequential instructions** in Linux v7.0-rc3.
- 2 The other **28 atomic-family instructions** left for future work.

Validation against the real kernel



浙江大学
ZHEJIANG UNIVERSITY

Inria

How do we gain confidence in our semantics?

Validation against the real kernel



How do we gain confidence in our semantics?

Ideal: prove equivalence with Linux C; this requires formalizing the kernel implementation.

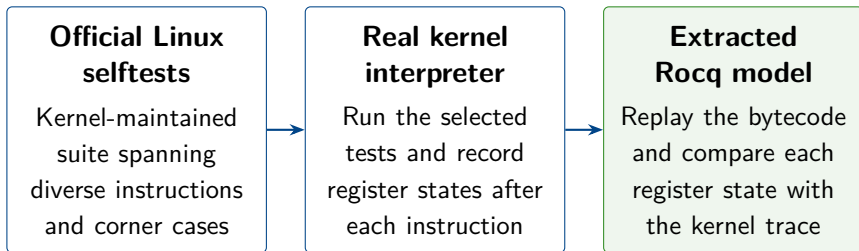
Validation against the real kernel



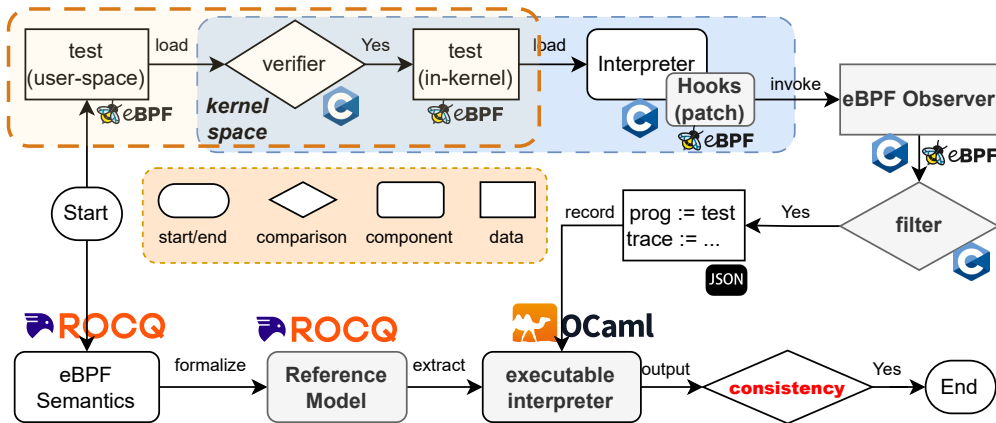
How do we gain confidence in our semantics?

Ideal: prove equivalence with Linux C; this requires formalizing the kernel implementation.

Instead: differential testing against the real kernel

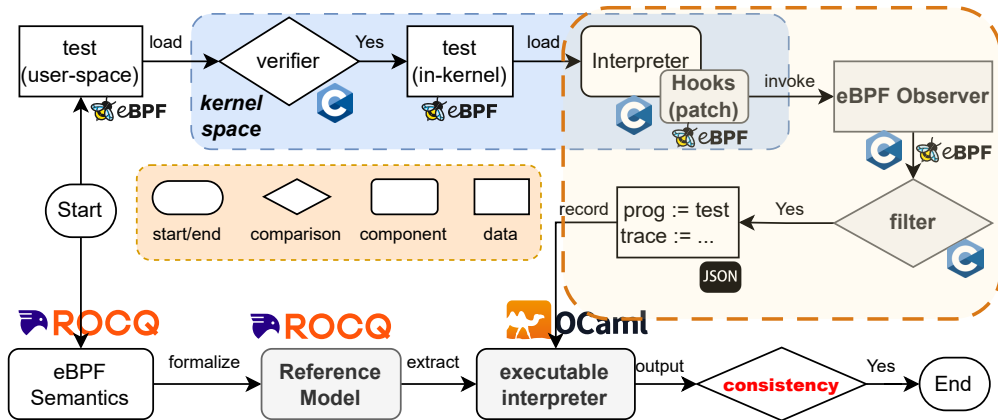


Validation framework



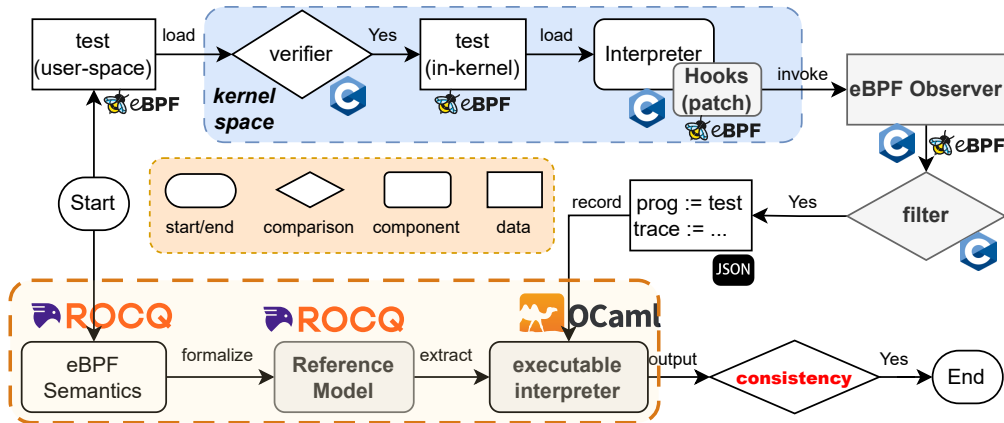
Official tests become verifier-processed in-kernel bytecode.

Validation framework



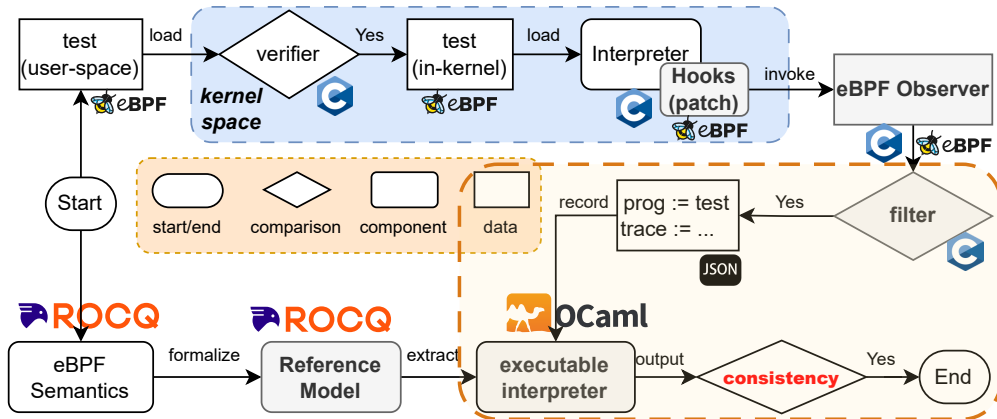
The kernel interpreter with BPF tracing hooks produces an execution trace.

Validation framework



Rocq semantics is extracted into an executable interpreter.

Validation framework



Replay the trace and compare register states step by step.

Validation framework (cont')



浙江大学
ZHEJIANG UNIVERSITY

Inria

Some technical details:

Validation framework (cont')



Some technical details:

- 1 Resolve nondeterminism in **external calls** by replaying the concrete outcomes recorded in the kernel trace, i.e. the return value in `r0`.

Validation framework (cont')



Some technical details:

- 1 Resolve nondeterminism in **external calls** by replaying the concrete outcomes recorded in the kernel trace, i.e. the return value in `r0`.
- 2 Validate only the **interpreter**; filter out tests involving **atomic instructions**, **privileged undefined behavior**, or cases that may **panic the kernel**.

*After filtering, **1,700+ tests** remain, covering every opcode in the modeled sequential ISA.*

Validation framework (cont')



Some technical details:

- ① Resolve nondeterminism in **external calls** by replaying the concrete outcomes recorded in the kernel trace, i.e. the return value in `r0`.
- ② Validate only the **interpreter**; filter out tests involving **atomic instructions**, **privileged undefined behavior**, or cases that may **panic the kernel**.

*After filtering, **1,700+ tests** remain, covering every opcode in the modeled sequential ISA.*

- ③ Compare only **register states** for simplicity³; any memory update that affects subsequent execution is eventually reflected in a register.

³Future work will compare the complete memory state.

Validation framework (cont')

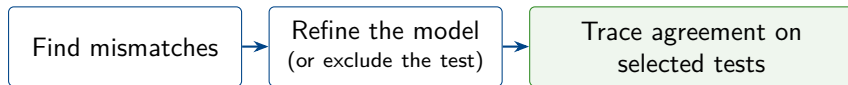


Some technical details:

- ① Resolve nondeterminism in **external calls** by replaying the concrete outcomes recorded in the kernel trace, i.e. the return value in `r0`.
- ② Validate only the **interpreter**; filter out tests involving **atomic instructions**, **privileged undefined behavior**, or cases that may **panic the kernel**.

*After filtering, **1,700+ tests** remain, covering every opcode in the modeled sequential ISA.*

- ③ Compare only **register states** for simplicity³; any memory update that affects subsequent execution is eventually reflected in a register.



³Future work will compare the complete memory state.

Application: What's the use of semantics?



The formal semantics can:

- ① Help **eBPF developers** gain a clearer understanding of instruction behavior.
- ② Provide a reusable mechanized semantic foundation for eBPF formal verification, especially for Rocq-based **research**.

Application: What's the use of semantics?

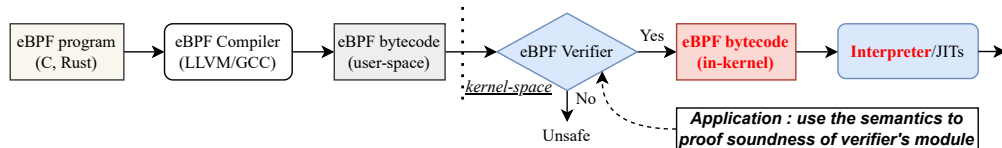


The formal semantics can:

- 1 Help **eBPF developers** gain a clearer understanding of instruction behavior.
- 2 Provide a reusable mechanized semantic foundation for eBPF formal verification, especially for Rocq-based **research**.

Application in this paper: **Formal verification of the eBPF verifier**

- 1 Verify the soundness of TNum module.
- 2 Contribute to verifier-related kernel development.



Application: Verifying TNum soundness



Tristate **N**umbers abstract domain,
to analyse bitwise uncertainty in eBPF Verifier.

- Binary Bit: 0, 1
- Tristate Bit: 0, 1, μ
(μ : not certain, could be 0 or 1)

Application: Verifying TNum soundness



Tristate **N**umbers abstract domain,
to analyse bitwise uncertainty in eBPF Verifier.

- Binary Bit: 0, 1
- Tristate Bit: 0, 1, μ
(μ : not certain, could be 0 or 1)

TNum example

$0t1\mu0\mu$ stands for:

$\{0b1000, 0b1001, 0b1100, 0b1101\}$

i.e. $\{8, 9, 12, 13\}$

Application: Verifying TNum soundness



Tristate **N**umbers abstract domain,
to analyse bitwise uncertainty in eBPF Verifier.

- Binary Bit: 0, 1
- Tristate Bit: 0, 1, μ
(μ : **not certain, could be 0 or 1**)

TNum example

$0t1\mu0\mu$ stands for:

$\{0b1000, 0b1001, 0b1100, 0b1101\}$

i.e. $\{8, 9, 12, 13\}$

Soundness: $mem(i, a) \wedge mem(j, b) \implies mem(op_{int}(i, j), op_{tnum}(a, b))$

Application: Verifying TNum soundness



Tristate Numbers abstract domain,
to analyse bitwise uncertainty in eBPF Verifier.

- Binary Bit: 0, 1
- Tristate Bit: 0, 1, μ
(μ : **not certain, could be 0 or 1**)

TNum example

$0t1\mu0\mu$ stands for:

$\{0b1000, 0b1001, 0b1100, 0b1101\}$

i.e. $\{8, 9, 12, 13\}$

Soundness: $mem(i, a) \wedge mem(j, b) \implies mem(op_{int}(i, j), op_{tnum}(a, b))$

Notations:

- Concrete values $i, j : int$. Abstract values $a, b : tnum$.
- $mem : int \times tnum \rightarrow bool$. $mem(i, a)$ holds iff $i \in \gamma(a)$, where γ is the **concretization function** of the TNum abstract domain.
- Concrete operator $op_{int} : int \times int \rightarrow int$.
Abstract operator $op_{tnum} : tnum \times tnum \rightarrow tnum$.

Application: Verifying TNum soundness



Tristate Numbers abstract domain,
to analyse bitwise uncertainty in eBPF Verifier.

- Binary Bit: 0, 1
- Tristate Bit: 0, 1, μ
(μ : **not certain, could be 0 or 1**)

TNum example

$0t1\mu0\mu$ stands for:

$\{0b1000, 0b1001, 0b1100, 0b1101\}$

i.e. $\{8, 9, 12, 13\}$

Soundness: $mem(i, a) \wedge mem(j, b) \implies mem(op_{int}(i, j), op_{tnum}(a, b))$

We proved TNum soundness for **all** ALU instructions in the eBPF ISA.

- **From kernel, unary:** neg, bswap16/32/64
- **From kernel, binary:** add, sub, mul, and, or, xor, lshift/rshift/arshift
- **From swb⁴, binary:** udiv/sdiv, umod/smod; also found some bugs in the code.
(Signed operations were completed after submission.)

⁴Fan et al. 2025. Program analysis combining generalized bit-level and word-level abstractions.
Proceedings of the ACM on Software Engineering, 2, ISSTA, (June 2025)

Application: Linux kernel contributions



We also contributed several patches to the upstream Linux kernel.

- ① `bpf: Reject negative offsets for ALU ops`
 - Bug found during formalization, merged in Oct 2025.
- ② `bpf: Add range tracking for BPF_DIV and BPF_MOD`
 - Enhancement with verified algorithm, merged in Jan 2026.
- ③ `bpf: Add bitwise tracking for BPF_END`
 - Enhancement with verified algorithm, merged in Feb 2026.
- ④ `bpf: Fix call offset truncation and OOB read in bpf_patch_call_args()`
 - Bug found during formalization, confirmed in Mar 2026, merged in May 2026.
- ⑤ ...

What Remains?



Concurrency

A full semantics for atomic instructions requires an **eBPF concurrency model**.

Maintainability

Keeping the model aligned with an **evolving Linux kernel** requires Rocq expertise.

Standardization

Keep the model and **specification** in sync, and try to contribute the spec to **the eBPF standard**.

What Remains?



Concurrency

A full semantics for atomic instructions requires an **eBPF concurrency model**.

Maintainability

Keeping the model aligned with an **evolving Linux kernel** requires Rocq expertise.

Standardization

Keep the model and **specification** in sync, and try to contribute the spec to **the eBPF standard**.

Follow-up direction

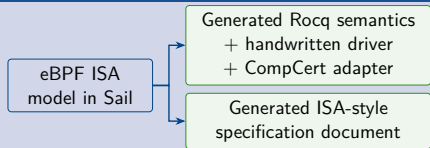
An **ISA-oriented source** that

- (1) is easier for ISA developers to update as Linux evolves.
- (2) can generate Rocq semantics together with an ISA-style spec.

Follow-up work: eBPF in Sail



One source, two artifacts



Instruction list: Linux 7.1

181 instructions from the interpreter

- 153 sequential instructions: executable semantics
- 28 atomic-family instructions: handled separately

Atomic handling

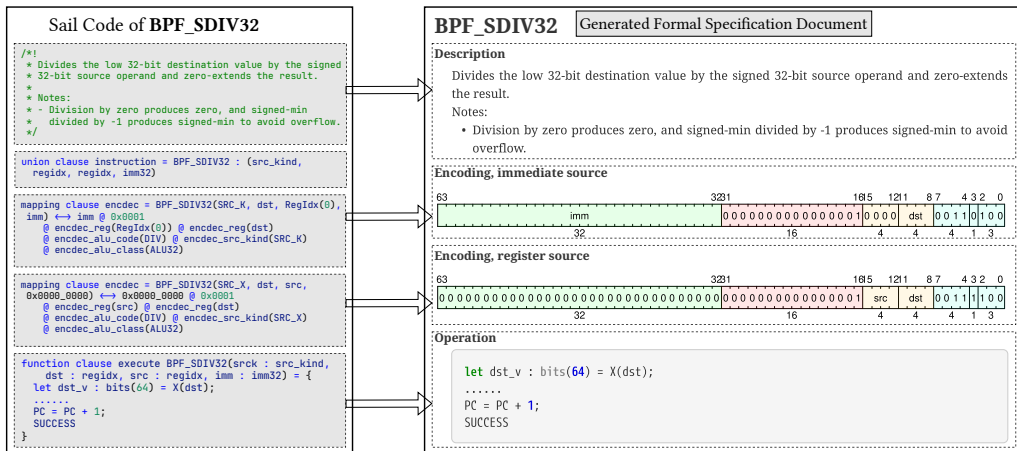
Two kinds: **atomic RMW** and **acquire/release forms**.

- Model encodings, value computation, and register writeback.
- Annotate memory effects; weak memory remains future work.

⁵Tang et al. 2026. Formal specification of linux ebpf instruction set architecture in sail. In *Proceedings of the 4th Workshop on EBPF and Kernel Extensions (eBPF'26)*, 26–32



From Sail to an ISA-style specification



Takeaways



Formal We mechanize all 153 sequential in-kernel eBPF instructions.

Faithful We validate the Rocq semantics against 1,700+ official Linux tests.

Useful The semantics supports proofs, bug finding, and upstream improvements.

Specified A handwritten specification links instructions to their semantics.

Next A maintainable Sail specification and proof-carrying code for eBPF.

Thank you! Questions?

Yazhou Tang tangyazhou@zju.edu.cn



Paper



Semantics & Proofs



Modified kernel